Run v51 (DRL-Swing-Options): Actor, Schedules, and the $\mathbb{E}[Q_T]$ Proxy

Scope. This note documents (i) the actor network architecture, (ii) the learning-rate and exploration-noise schedules, and (iii) the HHK-based approximation used for $\mathbb{E}[Q_T]$. Assumes `runv51.sh` hyperparameters and the current implementations in `src/networks.py`, `src/agent.py`, `src/swing_env.py`, `run.py`.

Run v51 essentials.

- Contract: K=1, T=0.0833, n=22, $(q_{\min}, q_{\max})$=(0, 2), $(Q_{\min}, Q_{\max})$=(0, 20), r=0.05, no refraction, c=0 (no convex cost).
- Actor/Critic: width 64, depth 2, activation SiLU, normalization RMSNorm, action head `tanh01`.
- LR: $(\eta_a, \eta_c) = (1.6{\times}10^{-4},\, 9.0{\times}10^{-5})$, warmup E_w=1024 episodes, cosine horizon E_h=40000, final fraction f=0.20, $\eta_{\min}$=10^{-6}.
- Exploration: pre-squash Gaussian noise with σ_0=1.30, floor $\sigma_{\min}$=0.26, plateau P=3200 episodes.

1. Actor network (`src/networks.py:145`)

State and action spaces. The environment observation contains (at least) the intrinsic value proxy

$s_t[0] = S_t - K$ and also S_t explicitly (see `src/swing_env.py`).

The actor outputs a normalized exercise intensity $a_t \in [0, 1]$, mapped to a contract quantity

$$q_t = q_{\min} + a_t\,(q_{\max} - q_{\min}) \quad \Rightarrow \quad \text{(v51) } q_t = 2a_t.$$

Architecture (v51). Let d_s be the state dimension and d_a=1 the action dimension. The actor is a 2-layer MLP with RMSNorm and SiLU:

$$h_1 = \mathrm{SiLU}(\mathrm{RMSNorm}(W_1 s + b_1)), \qquad h_2 = \mathrm{SiLU}(\mathrm{RMSNorm}(W_2 h_1 + b_2)),$$

$$z = W_3 h_2 + b_3, \qquad a^{\text{raw}} = \tfrac{1}{2}(\tanh(z) + 1) \in [0, 1].$$

Initialization. Hidden linear layers use orthogonal initialization (gain matched to activation), and the final linear layer is initialized uniformly in $[-3 \cdot 10^{-3},\, 3 \cdot 10^{-3}]$ (small initial actions).

Profitability gate (hard constraint + STE). The actor applies a hard gate which (forward-pass) forces $a_t = 0$ when the immediate profit is non-positive, but (backward-pass) uses a straight-through estimator (STE) so gradients flow as if ungated. With v51 settings (c=0, $q_{\min} \geq 0$, `tanh01` head), the gate simplifies to:

$$a_t = a_t^{\text{raw}}\,\mathbf{1}\{S_t - K > 0\} \quad \text{(forward, exactly)},$$

implemented as

$$a_t = a_t^{\text{raw}} + \left(a_t^{\text{raw}}\mathbf{1}\{S_t - K > 0\} - a_t^{\text{raw}}\right)_{\text{stop-grad}}.$$

The environment also masks actions to zero when the realized net payoff is ≤ 0 (`src/swing_env.py`), consistent with the gate when c=0.

2. LR decay (warmup + cosine) (`src/agent.py`)

In v51, actor and critic optimizers are AdamW with decoupled weight decay; the LR is controlled by a per-episode multiplicative factor $g(e)$ (scheduler step uses 1-based episode indexing):

$$\eta(e) = \eta_0\, g(e), \qquad e = 1, 2, \ldots$$

with

$$g(e) = \begin{cases} \dfrac{e}{E_w}, & e \leq E_w, \\ f + (1 - f)\,\dfrac{1 + \cos\!\left(\pi\,\frac{e - E_w}{E_h - E_w}\right)}{2}, & E_w < e < E_h, \\ f, & e \geq E_h, \end{cases} \qquad g(e) \leftarrow \max($$

Numerically (v51): E_w=1024, E_h=40000, f=0.20, $\eta_{\min}$=10^{-6}, $(\eta_{0,a}, \eta_{0,c}) = (1.6{\times}10^{-4},\, 9.0{\times}10^{-5})$. The floor is inactive since $\eta_{\min}/\eta_0 \ll f$ for both actor and critic.

3. Exploration noise decay (`src/agent.py:_p`

Noise is applied *to the actor pre-activation* z (before `tanh01` squashing and before the profitability gate):

$$z \leftarrow z + \sigma(e)\,\xi, \qquad \xi \sim \mathcal{N}(0, I).$$

The pre-squash standard deviation $\sigma(e)$ is:

$$\sigma(e) = \begin{cases} \sigma_0, & e < P, \\ \sigma_{\min} + \dfrac{\sigma_0 - \sigma_{\min}}{1 + \frac{e - P}{P}}, & e \geq P, \end{cases}$$

so the post-plateau decay is hyperbolic in e (half-life of order P). In v51: $(\sigma_0, \sigma_{\min}, P) = (1.30, 0.26, 3200)$, hence $\sigma(6400) \approx 0.26 + \frac{1.04}{2} = 0.78$.

4. Approximating $\mathbb{E}[Q_T]$ (`src/swing_env.py:4`

Objective. $Q_T := \sum_{k=1}^{n} q_k$ is the cumulative exercised volume. Under v51 constraints ($Q_{\min}$=0, no refraction, linear payoff/cost), the policy is close to "bang–bang": exercise $q_{\max}$ on in-the-money (ITM) dates until the global cap $Q_{\max}$ is reached. The helper `approximate_Q_T` returns a fast proxy for $\mathbb{E}[Q_T]$ used to initialize the actor head mean in `run.py:warmup_calibrate_actor_outputs`.

HHK model input. HHK is used in the form

$$S_t = \exp(f(t) + X_t + Y_t), \qquad Z_t := X_t + Y_t,$$

where X is Gaussian OU and Y is a mean-reverting jump component (see parameters in `runv51.sh`: α=12, σ=1.2, β=150, λ=6, μ_J=0.3, S_0=1).

Step A: marginal ITM probabilities. For each exercise date t_k (implemented on a grid $t_k = kT/n$), compute

$$p_k := \mathbb{P}(S_{t_k} > K) = \mathbb{P}\big(Z_{t_k} > \log K - f(t_k)\big).$$

If $\lambda \le 0$ (no jumps), Z_{t_k} is Gaussian and p_k is a normal tail probability. Otherwise, the code uses a Lugannani–Rice saddlepoint approximation: solve $K'(\hat\theta) = x$ for $x = \log K - f(t_k)$, where $K(\theta) = \log \mathbb{E}[e^{\theta Z_{t_k}}]$, then approximate $\mathbb{P}(Z_{t_k} \le x)$ by

$$\Phi(w)+\phi(w)\Big(\frac{1}{w}-\frac{1}{u}\Big), \quad w = \operatorname{sign}(\hat\theta)\sqrt{2(\hat\theta x - K(\hat\theta))}, \quad u = \hat\theta\sqrt{K''(\hat\theta)},$$

and return $p_k \approx 1 - \text{CDF}$.

Step B: dependence proxy via exchangeable indicators. Let $I_k = \mathbf{1}\{S_{t_k} > K\}$. The sum $N = \sum_{k=1}^{n} I_k$ is approximated by a Beta–Binomial distribution with:

$$\bar p = \frac{1}{n}\sum_{k=1}^{n} p_k, \qquad \rho_I \approx \operatorname{avg}_{i<j} \operatorname{Corr}(I_i, I_j).$$

The mapping $\operatorname{Corr}(Z_i, Z_j) \mapsto \operatorname{Corr}(I_i, I_j)$ is done via a Gaussian copula at thresholds $\tau_k = \Phi^{-1}(1 - p_k)$ using a bivariate normal CDF (sampling up to `max_pairs=4096` pairs). Then Beta–Binomial parameters are moment-matched:

$$s = \frac{1-\rho_I}{\rho_I}, \qquad a = \bar p\, s, \qquad b = (1-\bar p)\, s, \qquad N \sim \operatorname{BetaBin}(n, a, b).$$

Step C: truncated count-to-volume map. Define $m = \lfloor Q_{\max}/q_{\max} \rfloor$ and $q_{\text{last}} = Q_{\max} - m q_{\max}$. Then

$$\mathbb{E}[Q_T] \approx q_{\max}\, \mathbb{E}[\min(m, N)] + q_{\text{last}}\, \mathbb{P}(N > m),$$

computed by summing the Beta–Binomial pmf over $k = 0, \dots, n$ (and finally clipped to $[0, Q_{\max}]$). In v51, $q_{\max}$=2, $Q_{\max}$=20 so m=10 and q_{last}=0, i.e.

$$\mathbb{E}[Q_T] \approx 2\, \mathbb{E}[\min(10, N)].$$

Usage in initialization. `run.py` sets a target per-step volume $\mathbb{E}[Q_T]/n$ and calibrates the actor head so that the average normalized action is $\approx$ normalize$(\mathbb{E}[Q_T]/n)$; for v51 this is $(\mathbb{E}[Q_T]/n)/2$.